COMP5318 Assignment 1: Rice Classification

Group number: 74

Student 1 SID: 560457821

Student 2 SID: 530621856

Student 3 SID: 541006514

Student 4 SID: 550175951

1. Data Pre-processing

Data Loading

In this step, we load the dataset into the notebook using pandas. The dataset is expected to be in the same directory as this notebook, or the correct relative path should be provided.

To ensure that the code works on any system, we avoid using absolute file paths and instead rely on relative paths. This allows the tutor to run the notebook without modifying file locations.

The dataset used in this assignment is `rice-final2.csv`, where each row represents a rice grain sample and the last column represents the class label.

In [1]:
```
# Import all libraries
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import  MinMaxScaler
from sklearn.model_selection import StratifiedKFold
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import cross_val_score
from sklearn.metrics import accuracy_score, f1_score
from sklearn.ensemble import AdaBoostClassifier, RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.svm import SVC
```

In [2]:
```
# Ignore future warnings
from warnings import simplefilter
simplefilter(action='ignore', category=FutureWarning)
```

File Paths and Project Structure

To ensure portability and ease of execution, this notebook uses **relative file paths** instead of absolute paths.

Project Directory Layout

```
COMP5318_Assignment/
│
├── rice-final2.csv          # Main dataset
├── test-before.csv          # Test dataset
├── comp5318-a1-2026-template.ipynb   # Notebook
```

Description

- All files are stored in a single directory.
- The notebook reads datasets using relative paths (e.g., `pd.read_csv("rice-final2.csv")`).
- This allows the notebook to run seamlessly on different systems without requiring path modifications.

Requirement

Ensure that the dataset files are placed in the same directory as the notebook before execution.

In [3]:
```
# Load the rice dataset: rice-final2.csv
df = pd.read_csv("rice-final2.csv", na_values="?")
df.isnull().sum()
df.shape
df.head()
```

Out[3]:

	Area	Perimiter	Major_Axis_Length	Minor_Axis_Length	Eccentricity	Convex_Area
0	12573.0	461.466003	192.903351	84.572075	0.898772	12893.0
1	12845.0	464.121002	194.332214	85.524338	0.897952	13125.0
2	14055.0	488.748993	207.751755	87.250328	0.907536	14484.0
3	14412.0	490.324005	207.476135	89.689514	0.901735	14703.0
4	14658.0	477.117004	189.566635	99.997780	0.849551	15048.0

Data Preprocessing

Before applying machine learning models, the dataset must be cleaned and prepared.

The following preprocessing steps are performed:

- Missing values are replaced using the mean of each column.
- Feature values are normalized to the range [0, 1] using Min-Max scaling.
- Class labels are converted from categorical values (`class1` , `class2`) to numerical values (0, 1).

These steps ensure that the data is consistent and suitable for machine learning algorithms.

In [4]:
```
# Pre-process dataset

X = df.iloc[:,:-1]
y = df.iloc[:,-1]

# Fixing missing values by replacing the missing values with the mean of the column
imputer = SimpleImputer( missing_values = np.nan , strategy="mean")
X = imputer.fit_transform(X)

# Normalize between 0 and 1

scaler = MinMaxScaler()
X = scaler.fit_transform(X)

# Change Class 1 to 0 and Class 2 to 1
y = np.where(y=="class1" ,0, 1)

print(X.shape)
print(y.shape)
```

```
(1400, 7)
(1400,)
```

In [5]:
```
# Print first ten rows of pre-processed dataset to 4 decimal places as per assignme
# A function is provided to assist

def print_data(X, y, n_rows=10):
    """Takes a numpy data array and target and prints the first ten rows.

    Arguments:
        X: numpy array of shape (n_examples, n_features)
        y: numpy array of shape (n_examples)
        n_rows: numpy of rows to print
    """
    for example_num in range(n_rows):
        for feature in X[example_num]:
            print("{:.4f}".format(feature), end=",")

        if example_num == len(X)-1:
            print(y[example_num],end="")
```

```
        else:
            print(y[example_num])
```

Data Printing

After preprocessing, the first 10 rows of the dataset are printed to verify that all transformations have been applied correctly.

The feature values are formatted to 4 decimal places as required, and the class labels are displayed as integers. This helps ensure that the data is clean and ready for model training.

In [6]:
```
print_data(X,y)
```

```
0.4628,0.5406,0.5113,0.4803,0.7380,0.4699,0.1196,1
0.4900,0.5547,0.5266,0.5018,0.7319,0.4926,0.8030,1
0.6109,0.6847,0.6707,0.5409,0.8032,0.6253,0.1185,0
0.6466,0.6930,0.6677,0.5961,0.7601,0.6467,0.2669,0
0.6712,0.6233,0.4755,0.8293,0.3721,0.6803,0.4211,1
0.2634,0.2932,0.2414,0.4127,0.5521,0.2752,0.2825,1
0.8175,0.9501,0.9515,0.5925,0.9245,0.8162,0.0000,0
0.3174,0.3588,0.3601,0.3908,0.6921,0.3261,0.8510,1
0.3130,0.3050,0.2150,0.5189,0.3974,0.3159,0.4570,1
0.5120,0.5237,0.4409,0.6235,0.5460,0.5111,0.3155,1
```

2. Build Classifiers

- Part 1: Logistic Regression, Naïve Bayes
- Part 2: KNN, Decision Tree, Ada Boost, Gradient Boost, Random Forest, SVM

Part 1: Cross-validation without parameter tuning

Cross-Validation Setup

We use stratified 10-fold cross-validation to evaluate model performance.

Stratified sampling ensures that each fold maintains the same class distribution as the original dataset. This provides a more reliable evaluation compared to simple random splitting.

The parameter `random_state=0` is used to ensure reproducibility of results.

In [7]:
```
## Setting the 10 fold stratified cross-validation
cvKFold=StratifiedKFold(n_splits=10, shuffle=True, random_state=0)

# The stratified folds from cvKFold should be provided to the classifiers
```

Logistic Regression

Logistic Regression is a supervised classification algorithm used to predict binary outcomes.

In this step:

- The Logistic Regression model is initialized.
- The model is evaluated using stratified 10-fold cross-validation.
- The average accuracy across all folds is computed.

This provides a reliable estimate of the model's performance on unseen data.

In [8]:
```
# Logistic Regression
lr_model = LogisticRegression(random_state=0 , max_iter = 1000)

lr_scores = cross_val_score( lr_model, X, y , cv=cvKFold, scoring="accuracy")
lr_mean = lr_scores.mean()
print("LogR average cross-validation accuracy: {:.4f}".format(lr_mean))
```

```
LogR average cross-validation accuracy: 0.9386
```

Naive Bayes

Naive Bayes is a probabilistic classification algorithm based on Bayes' theorem. It assumes that features are independent of each other.

In this step:

- A Gaussian Naive Bayes model is initialized.
- The model is evaluated using stratified 10-fold cross-validation.
- The average accuracy is calculated.

This allows us to compare its performance with Logistic Regression.

In [9]:
```
# Naïve Bayes

nb_model =GaussianNB()
nb_scores = cross_val_score(nb_model, X, y, cv= cvKFold, scoring= "accuracy")

nb_mean = nb_scores.mean()
print("NB average cross-validation accuracy: {:.4f}".format(nb_mean))
```

```
NB average cross-validation accuracy: 0.9264
```

Part 1 Results

Results

The performance of both models is reported using average cross-validation accuracy.

The results are formatted to four decimal places as required. These results help compare the effectiveness of different classifiers on the dataset.

In [10]:
```
# Print results for each classifier in part 1 to 4 decimal places here:
print("LogR average cross-validation accuracy: {:.4f}".format(lr_mean))
print("NB average cross-validation accuracy: {:.4f}".format(nb_mean))
```

```
LogR average cross-validation accuracy: 0.9386
NB average cross-validation accuracy: 0.9264
```

Part 2: Cross-validation with parameter tuning

K-Nearest Neighbour (KNN)

In this section, we apply the K-Nearest Neighbour (KNN) classifier. We use GridSearchCV with 10-fold stratified cross-validation to find the optimal number of neighbors. The best parameters, best cross-validation accuracy, and test accuracy are reported.

In [11]:
```
# KNN implementation
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    stratify=y,
    random_state=0
)

# Define model
knn = KNeighborsClassifier()

# Parameter grid
param_grid_knn = {
    'n_neighbors': [3, 5, 7, 9, 11]
}

# GridSearch with 10-fold CV
grid_knn = GridSearchCV(
    estimator=knn,
    param_grid=param_grid_knn,
    cv=cvKFold,
    scoring='accuracy',
    n_jobs=-1
)

# Fit model
grid_knn.fit(X_train, y_train)

# Predict
y_pred_knn = grid_knn.predict(X_test)
```

```
# Print results
print("KNN")
print("Best parameters:", grid_knn.best_params_)
print("Best cross-validation accuracy: {:.4f}".format(grid_knn.best_score_))
print("Test accuracy: {:.4f}".format(accuracy_score(y_test, y_pred_knn)))
```

```
KNN
Best parameters: {'n_neighbors': 11}
Best cross-validation accuracy: 0.9384
Test accuracy: 0.9286
```

Decision Tree (DT)

In this section, we apply the Decision Tree classifier. We use GridSearchCV with 10-fold stratified cross-validation to tune the hyperparameters. The best parameters, best cross-validation accuracy, and test accuracy are reported.

In [12]:
```
# Decision Tree implementation

# Define model
dt = DecisionTreeClassifier(random_state=0)

# Parameter grid
param_grid_dt = {
    'max_depth': [None, 5, 10, 15],
    'min_samples_split': [2, 5, 10]
}

# GridSearch with 10-fold CV
grid_dt = GridSearchCV(
    estimator=dt,
    param_grid=param_grid_dt,
    cv=cvKFold,
    scoring='accuracy',
    n_jobs=-1
)

# Fit model
grid_dt.fit(X_train, y_train)

# Predict
y_pred_dt = grid_dt.predict(X_test)

# Print results
print("Decision Tree")
print("Best parameters:", grid_dt.best_params_)
print("Best cross-validation accuracy: {:.4f}".format(grid_dt.best_score_))
print("Test accuracy: {:.4f}".format(accuracy_score(y_test, y_pred_dt)))
```

```
Decision Tree
Best parameters: {'max_depth': 5, 'min_samples_split': 2}
Best cross-validation accuracy: 0.9330
Test accuracy: 0.9214
```

ADABOOST WITH HYPERPARAMETER TUNING

In this section, we implement the AdaBoost ensemble classifier. AdaBoost works by combining multiple "weak learners" to create a single strong learner. It does this by training models sequentially, where each new model focuses on correcting the errors made by the previous one.

In [13]:
```
# Ada Boost
# 1) Split the data into 80% training and 20% test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,
    random_state=0
)


# 2) Set up 10-fold stratified cross-validation
cvKFold = StratifiedKFold(n_splits=10, shuffle=True, random_state=0)


# 3) AdaBoost Classifier + Grid Search
ada_clf = AdaBoostClassifier(random_state=0)

# Parameter grid
ada_param_grid = {
    'n_estimators': [50, 100, 150, 200],
    'learning_rate': [0.1, 0.3, 0.5, 1.0]
}

# GridSearch with 10-fold CV
ada_grid = GridSearchCV(
    estimator=ada_clf,
    param_grid=ada_param_grid,
    cv=cvKFold,
    scoring='accuracy',
    n_jobs=-1
)

ada_grid.fit(X_train, y_train)

# Best AdaBoost model and predictions on the test set
best_ada = ada_grid.best_estimator_
y_pred_ada = best_ada.predict(X_test)

ada_best_cv_acc = ada_grid.best_score_
ada_test_acc = accuracy_score(y_test, y_pred_ada)

#print result
print("ADABOOST RESULTS:")
print("Best parameters:", ada_grid.best_params_)
print("Best cross validation accuracy: {:.4f}".format(ada_best_cv_acc))
print("Test set accuracy: {:.4f}".format(ada_test_acc))
```

```
ADABOOST RESULTS:
Best parameters: {'learning_rate': 1.0, 'n_estimators': 150}
Best cross validation accuracy: 0.9455
Test set accuracy: 0.9393
```

Gradient Boosting Classifier

Gradient Boosting builds an ensemble of shallow decision trees sequentially.
Each new tree corrects the errors of the previous ones, making it powerful for tabular data.

Hyperparameters being tuned:

Parameter	Values Tried	What it controls
`n_estimators`	50, 100, 150	Number of trees in the ensemble
`learning_rate`	0.1, 0.2, 0.3, 0.5	How much each tree contributes — smaller = more conservative
`max_depth`	1, 3, 5, 7	Maximum depth of each individual tree — controls complexity

Total combinations: 3 × 4 × 4 = **48 parameter combinations**, each evaluated with 10-fold CV.

In [14]:
```
# Gradient Boost

# ── Train / Validation split ──────────────────────────────────────────
# 80% training data for GridSearchCV, 20% held-out validation for final evaluation
# stratify=y ensures both splits have the same class distribution
X_train, X_val, y_train, y_val = train_test_split(
    X, y,
    test_size=0.2,       # 20% of data reserved for validation
    stratify=y,          # preserve class ratio in both splits
    random_state=0       # reproducibility
)

# ── Gradient Boosting Hyperparameter Grid ─────────────────────────────
# GridSearchCV will try every combination of these values
gb_params = {
    'n_estimators':  [50, 100, 150],         # number of boosting rounds (trees)
    'learning_rate': [0.1, 0.2, 0.3, 0.5],  # step size shrinkage
    'max_depth':     [1, 3, 5, 7],           # depth of each tree (1=stump)
}

# ── Grid Search with 10-fold Stratified CV ────────────────────────────
# n_jobs=-1 uses all CPU cores in parallel to speed up the search
gb_search = GridSearchCV(
    GradientBoostingClassifier(random_state=0),
    gb_params,
    cv=cvKFold,           # the same 10-fold CV used throughout
    scoring='accuracy',   # optimise for accuracy
    n_jobs=-1,            # parallelise across all CPU cores
)
```

```
# Fit GridSearchCV on training data — finds the best hyperparameter combination
gb_search.fit(X_train, y_train)

# Extract the best model (already refitted on full X_train with best params)
best_gb = gb_search.best_estimator_
print('Best GB params found:', gb_search.best_params_)

# Best mean CV accuracy across 10 folds (on training data)
print('Gradient Boost CV accuracy: {:.4f}'.format(gb_search.best_score_))

# Generate predictions on the held-out validation set
y_val_gb = best_gb.predict(X_val)

# Validation set metrics — unbiased estimate of real-world performance
print('Gradient Boost validation accuracy:  {:.4f}'.format(accuracy_score(y_val, y_
print('Gradient Boost validation macro F1:  {:.4f}'.format(f1_score(y_val, y_val_gb
print('Gradient Boost validation weighted F1:{:.4f}'.format(f1_score(y_val, y_val_g
```

```
Best GB params found: {'learning_rate': 0.1, 'max_depth': 1, 'n_estimators': 50}
Gradient Boost CV accuracy: 0.9446
Gradient Boost validation accuracy:  0.9429
Gradient Boost validation macro F1:  0.9414
Gradient Boost validation weighted F1:0.9427
```

Random Forest Classification with Hyperparameter Tuning

In this section, we use a Random Forest classifier, which is an ensemble learning method that builds multiple decision trees simultaneously. Unlike AdaBoost, which builds trees one after another, Random Forest builds them in parallel. The final classification is determined by a "majority vote" from all the individual trees, which makes the model very robust against noise and outliers.

In [15]:

```
# Random Forest

#Random Forest Classifier
rf_clf = RandomForestClassifier(
    criterion='entropy',
    max_features='sqrt',
    random_state=0
)

#parameter Grid
rf_param_grid = {
    'n_estimators': [10, 30, 60, 100, 200],
    'max_leaf_nodes': [6, 12, 18, 24]
}

# GridSearch with 10-fold CV
rf_grid = GridSearchCV(
    estimator=rf_clf,
    param_grid=rf_param_grid,
    cv=cvKFold,
    scoring='accuracy',
```

```
    n_jobs=-1
)

rf_grid.fit(X_train, y_train)

# Best Random Forest model and predictions on the test set
best_rf = rf_grid.best_estimator_
y_pred_rf = best_rf.predict(X_test)

rf_best_cv_acc = rf_grid.best_score_
rf_test_acc = accuracy_score(y_test, y_pred_rf)
rf_test_f1_macro = f1_score(y_test, y_pred_rf, average='macro')
rf_test_f1_weighted = f1_score(y_test, y_pred_rf, average='weighted')

#Print Result
print("\nRANDOM FOREST RESULTS:")
print("Best parameters:", rf_grid.best_params_)
print("Best cross-validation accuracy: {:.4f}".format(rf_best_cv_acc))
print("Test set accuracy: {:.4f}".format(rf_test_acc))
print("Test set macro F1: {:.4f}".format(rf_test_f1_macro))
print("Test set weighted F1: {:.4f}".format(rf_test_f1_weighted))
```

```
RANDOM FOREST RESULTS:
Best parameters: {'max_leaf_nodes': 18, 'n_estimators': 30}
Best cross-validation accuracy: 0.9437
Test set accuracy: 0.9357
Test set macro F1: 0.9341
Test set weighted F1: 0.9356
```

Support Vector Machine (SVM)

SVM finds the maximum-margin hyperplane that best separates the two classes.
The **RBF (Radial Basis Function) kernel** maps features into a higher-dimensional space, allowing SVM to capture non-linear decision boundaries.

Hyperparameters being tuned:

Parameter	Values Tried	What it controls
`C`	0.01, 0.1, 1, 5	Regularisation — low C = wider margin (more misclassifications allowed); high C = strict boundary
`gamma`	0.01, 0.1, 1, 10	RBF kernel width — low gamma = broad influence; high gamma = tight fit to training points
`kernel`	rbf	Kernel function — RBF handles non-linear boundaries

Total combinations: 4 × 4 × 1 = **16 combinations** × 10 folds = 160 model fits.

> SVM is sensitive to feature scale — this is why MinMaxScaler was applied during pre-processing.

In [16]:
```
# ── SVM Hyperparameter Grid ──────────────────────────────────────────────
svc_params = {
    'C':      [0.01, 0.1, 1, 5],     # regularisation strength
    'gamma':  [0.01, 0.1, 1, 10],    # RBF kernel coefficient
    'kernel': ['rbf'],               # radial basis function kernel
}

# ── Grid Search with 10-fold Stratified CV ──────────────────────────────
svc_search = GridSearchCV(
    SVC(random_state=0),
    svc_params,
    cv=cvKFold,           # same CV splits as all other models
    scoring='accuracy',   # optimise for accuracy
    n_jobs=-1,            # use all CPU cores
)

# Fit on training data — exhaustive search over all param combinations
svc_search.fit(X_train, y_train)

# Extract the best model
best_svc = svc_search.best_estimator_
print('Best SVM params found:', svc_search.best_params_)

# Best mean CV accuracy across 10 folds (on training data)
print('SVM CV accuracy: {:.4f}'.format(svc_search.best_score_))

# Generate predictions on the held-out validation set
y_val_svc = best_svc.predict(X_val)

# Validation set metrics
print('SVM validation accuracy:  {:.4f}'.format(accuracy_score(y_val, y_val_svc)))
print('SVM validation macro F1:  {:.4f}'.format(f1_score(y_val, y_val_svc, average=
print('SVM validation weighted F1:{:.4f}'.format(f1_score(y_val, y_val_svc, average
```

```
Best SVM params found: {'C': 5, 'gamma': 1, 'kernel': 'rbf'}
SVM CV accuracy: 0.9429
SVM validation accuracy:  0.9321
SVM validation macro F1:  0.9305
SVM validation weighted F1:0.9320
```

Part 2: Results

In [17]:
```
# Perform Grid Search with 10-fold stratified cross-validation (GridSearchCV in skl
# The stratified folds from cvKFold should be provided to GridSearchV

# This should include using train_test_split from sklearn.model_selection with stra
# Print results for each classifier here. All the reported results should be printe

# example printing:
print("\nKNN RESULTS:")
print("Best parameters:", grid_knn.best_params_)
print("Best CV accuracy: {:.4f}".format(grid_knn.best_score_))
print("Test accuracy: {:.4f}".format(accuracy_score(y_test, y_pred_knn)))

print("\nDECISION TREE RESULTS:")
```

```
print("Best parameters:", grid_dt.best_params_)
print("Best CV accuracy: {:.4f}".format(grid_dt.best_score_))
print("Test accuracy: {:.4f}".format(accuracy_score(y_test, y_pred_dt)))

print("\nGRADIENT BOOST RESULTS")
print('Gradient Boost best params:', gb_search.best_params_)
print('Gradient Boost CV accuracy: {:.4f}'.format(gb_search.best_score_))
print('Gradient Boost validation accuracy:  {:.4f}'.format(accuracy_score(y_val, y_
print('Gradient Boost validation macro F1:  {:.4f}'.format(f1_score(y_val, y_val_gb
print('Gradient Boost validation weighted F1:{:.4f}'.format(f1_score(y_val, y_val_g

print("\nSVM RESULTS:")
print('SVM best params:', svc_search.best_params_)
print('SVM CV accuracy: {:.4f}'.format(svc_search.best_score_))
print('SVM validation accuracy:  {:.4f}'.format(accuracy_score(y_val, y_val_svc)))
print('SVM validation macro F1:  {:.4f}'.format(f1_score(y_val, y_val_svc, average=
print('SVM validation weighted F1:{:.4f}'.format(f1_score(y_val, y_val_svc, average


print("\nADABOOST RESULTS:")
print("Best parameters:", ada_grid.best_params_)
print("Best cross validation accuracy: {:.4f}".format(ada_best_cv_acc))
print("Test set accuracy: {:.4f}".format(ada_test_acc))

print("\nRANDOM FOREST RESULTS:")
print("Best parameters:", rf_grid.best_params_)
print("Best cross-validation accuracy: {:.4f}".format(rf_best_cv_acc))
print("Test set accuracy: {:.4f}".format(rf_test_acc))
print("Test set macro F1: {:.4f}".format(rf_test_f1_macro))
print("Test set weighted F1: {:.4f}".format(rf_test_f1_weighted))
```

```
KNN RESULTS:
Best parameters: {'n_neighbors': 11}
Best CV accuracy: 0.9384
Test accuracy: 0.9286

DECISION TREE RESULTS:
Best parameters: {'max_depth': 5, 'min_samples_split': 2}
Best CV accuracy: 0.9330
Test accuracy: 0.9214

GRADIENT BOOST RESULTS
Gradient Boost best params: {'learning_rate': 0.1, 'max_depth': 1, 'n_estimators': 5
0}
Gradient Boost CV accuracy: 0.9446
Gradient Boost validation accuracy:  0.9429
Gradient Boost validation macro F1:  0.9414
Gradient Boost validation weighted F1:0.9427

SVM RESULTS:
SVM best params: {'C': 5, 'gamma': 1, 'kernel': 'rbf'}
SVM CV accuracy: 0.9429
SVM validation accuracy:  0.9321
SVM validation macro F1:  0.9305
SVM validation weighted F1:0.9320

ADABOOST RESULTS:
Best parameters: {'learning_rate': 1.0, 'n_estimators': 150}
Best cross validation accuracy: 0.9455
Test set accuracy: 0.9393

RANDOM FOREST RESULTS:
Best parameters: {'max_leaf_nodes': 18, 'n_estimators': 30}
Best cross-validation accuracy: 0.9437
Test set accuracy: 0.9357
Test set macro F1: 0.9341
Test set weighted F1: 0.9356
```

Test your code

Testing on Additional Dataset

To ensure that the implemented pipeline is general and not hardcoded for a specific dataset, we test the code on an additional dataset (`test-before.csv`).

The same preprocessing steps are applied, and the models are evaluated again. This step verifies that the code works correctly on datasets with different structures.

Note: The results from this section are not included in the final evaluation, as they are only used for testing purposes.

In [18]:
```
#load the test dataset to test out your model
```

```
test_df = pd.read_csv("test-before.csv", na_values="?")

X_test = test_df.iloc[:, :-1].values
y_test = test_df.iloc[:, -1].values

# Fit the Imputer again since the test data set has mismatches with the no. or feat
imputer = SimpleImputer(strategy="mean")
X_test = imputer.fit_transform(X_test)

scaler = MinMaxScaler()
X_test = scaler.fit_transform(X_test)
y_test = np.where(y_test == "class1", 0, 1)


# Evaluate
lr_scores_test = cross_val_score(lr_model, X_test, y_test, cv=cvKFold, scoring='acc
nb_scores_test = cross_val_score(nb_model, X_test, y_test, cv=cvKFold, scoring='acc
ada_scores_test = cross_val_score(best_ada, X_test, y_test, cv=cvKFold, scoring='ac
rf_scores_test = cross_val_score(best_rf, X_test, y_test, cv=cvKFold, scoring='accu
knn_scores_test = cross_val_score(grid_knn.best_estimator_, X_test, y_test, cv=cvKF
dt_scores_test = cross_val_score(grid_dt.best_estimator_, X_test, y_test, cv=cvKFol
gb_scores_test = cross_val_score(best_gb, X_test, y_test, cv=cvKFold, scoring='accu
svm_scores_test = cross_val_score(best_svc, X_test, y_test, cv=cvKFold, scoring='ac

# print("===== TEST RESULTS =====")

# print(lr_scores_test.mean())
# print(nb_scores_test.mean())
# print("AdaBoost average accuracy: {:.4f}".format(ada_scores_test.mean()))
# print("Random Forest average accuracy: {:.4f}".format(rf_scores_test.mean()))
# print("KNN: {:.4f}".format(knn_scores_test.mean()))
# print("Decision Tree: {:.4f}".format(dt_scores_test.mean()))
# print("Gradient Boosting: {:.4f}".format(gb_scores_test.mean()))
# print("SVM: {:.4f}".format(svm_scores_test.mean()))
```

3. Reflection and Discussion

Part 1 – Logistic Regression and Naive Bayes

Model Performance

Logistic Regression achieved an average cross-validation accuracy of **0.9386**, while Naive Bayes achieved **0.9264**.

Analysis

The better performance of Logistic Regression can be attributed to its ability to model relationships between features, whereas Naive Bayes assumes that all features are independent. In this dataset, features such as area and perimeter are likely correlated, which negatively impacts the performance of Naive Bayes.

Model Comparison

- **Logistic Regression**: More flexible, captures feature relationships, higher accuracy
- **Naive Bayes**: Simpler, faster, but assumes feature independence

---

Part 2 – Advanced Models and Hyperparameter Tuning

Model Performance

In Part 2, multiple models including K-Nearest Neighbour (KNN), Decision Tree, Gradient Boosting, Support Vector Machine (SVM), AdaBoost, and Random Forest were evaluated using hyperparameter tuning with GridSearchCV and 10-fold stratified cross-validation.

- **KNN** achieved a best cross-validation accuracy of **0.9384** and test accuracy of **0.9286**
- **Decision Tree** achieved a best cross-validation accuracy of **0.9330** and test accuracy of **0.9214**
- **AdaBoost** achieved a best cross-validation accuracy of **0.9455**
- **Random Forest** achieved a best cross-validation accuracy of **0.9437**
- Gradient Boosting and SVM also demonstrated strong performance after tuning

---

Model Analysis

- **KNN** benefits from normalized feature space and captures local data structure effectively but is sensitive to the choice of neighbors and computationally expensive for large datasets.

- **Decision Tree** is easy to interpret and can capture non-linear relationships, but is prone to overfitting if depth is not controlled.

- **Gradient Boosting** builds trees sequentially, where each tree corrects errors of the previous one. Lower learning rates combined with more estimators improve generalization.

- **SVM (RBF Kernel)** maps features into higher-dimensional space to separate non-linear classes. It is sensitive to feature scaling and parameters such as **C** and **gamma** significantly affect performance.

- **AdaBoost** improves performance by combining weak learners sequentially and focusing on misclassified samples.

- **Random Forest** reduces overfitting by averaging multiple decision trees and introduces randomness through feature selection and tree structure.

Impact of Hyperparameter Tuning

Hyperparameter tuning using GridSearchCV significantly improved model performance by systematically searching for optimal parameter values.

- For **KNN**, tuning the number of neighbors helped balance bias and variance.
- For **Decision Tree**, parameters such as max_depth and min_samples_split controlled overfitting.
- For **AdaBoost**, increasing the number of estimators improved learning of complex patterns.
- For **Random Forest**, limiting max_leaf_nodes helped prevent overfitting while maintaining strong performance.
- For **SVM**, tuning **C** and **gamma** improved the decision boundary and generalization.

General Observations

- Cross-validation ensured that models generalized well and were not overfitted to a specific split of the data.
- Normalization was crucial for distance-based and kernel-based models such as KNN and SVM.
- Ensemble models such as Random Forest, Gradient Boosting, and AdaBoost achieved higher accuracy due to their ability to combine multiple learners.
- Simpler models such as Logistic Regression still performed competitively while being more interpretable.

Conclusion

Overall, ensemble methods such as AdaBoost and Random Forest achieved the best performance, while Logistic Regression provided a strong baseline with good interpretability. The choice of model ultimately depends on the trade-off between accuracy, computational cost, and interpretability.

Proper preprocessing and hyperparameter tuning played a crucial role in improving model performance and ensuring reliable generalization on unseen data.

AI Acknowledgement

The following AI tools were used to assist in this assignment:

- **ChatGPT (OpenAI)**: Used for understanding machine learning concepts, clarifying doubts related to preprocessing, Model understanding and assisting in debugging code and improving explanations.
- **Claude (Anthropic)**: Used for reviewing code, identifying potential issues (such as preprocessing in the test dataset), and suggesting improvements.

These tools were used only as learning aids and for guidance. All code implementation, model development, analysis, and final decisions were carried out independently by the authors.

All outputs generated with the assistance of AI were carefully reviewed, validated, and modified where necessary to ensure correctness and compliance with academic integrity guidelines.